

SPECIFICATION

CareConnect

AI-Powered Patient Follow-Up and Reminder Management System

Document Type	Software Requirements Specification (SRS)
Version	1.0
Project	CareConnect — AI Patient Follow-Up Assistant
Domain	Healthcare
Theme	Theme 9 — IIT Roorkee AIOps Capstone
Team	Team 11
Authors	Manisha Pradhan, Sunit Mukherjee, Reffel Murzello, Vivek Sharma, Pradeep Kumar M, Mohammed Rohith
Date	June 2026
Status	Final

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions and Acronyms
- 1.4 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Assumptions and Constraints

3. Functional Requirements

- 3.1 Patient Data Management
- 3.2 AI Agent Workflow
- 3.3 RAG Pipeline
- 3.4 Reminder Generation
- 3.5 Escalation and Human Approval
- 3.6 Dashboard and Frontend

4. Non-Functional Requirements

- 4.1 Performance
- 4.2 Reliability and Availability
- 4.3 Security
- 4.4 Usability
- 4.5 Maintainability

5. System and Data Requirements

- 5.1 Hardware Requirements
- 5.2 Software Dependencies
- 5.3 Data Requirements

6. Use Cases

7. User Stories

8. Integration Requirements

9. Constraints and Limitations

10. Glossary

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for CareConnect, an AI-powered patient follow-up and reminder management system. The document serves as the authoritative reference for design, implementation, testing, and evaluation of the system.

The target audience for this document includes the development team (Team 11), project evaluators at IIT Roorkee, and any future developers who extend or maintain the system.

1.2 Scope

CareConnect is a healthcare-domain AI application designed to automate and improve post-discharge patient follow-up communications in hospital settings. The system:

- Ingests synthetic patient records, appointment data, and follow-up histories from CSV datasets
- Runs a multi-agent LangGraph workflow to summarize patient status and classify clinical risk
- Retrieves relevant follow-up protocols from a ChromaDB vector store using Retrieval-Augmented Generation (RAG)
- Generates personalized follow-up reminders in three channels: SMS, Email, and Phone Call Script
- Escalates high-risk and critical patients to a human clinician via an in-loop approval mechanism
- Presents all outputs through a Streamlit web dashboard accessible via AWS EC2

The system is built as a capstone project for the Advanced PG Certification in AI Engineering on Cloud and AIOps at IIT Roorkee (powered by Futureense) and is intended to demonstrate enterprise-grade agentic AI capabilities.

1.3 Definitions and Acronyms

Term / Acronym	Definition
AI	Artificial Intelligence
LLM	Large Language Model — a deep learning model trained on large text corpora for natural language
GPT-4o	OpenAI's GPT-4 Omni model — the LLM used for all agent reasoning in this project
RAG	Retrieval-Augmented Generation — technique combining vector search with LLM generation
LangGraph	Python library for building stateful, multi-agent LLM workflows as directed graphs
LangChain	Python library providing document loaders, text splitters, and retrieval chains
ChromaDB	Open-source vector database for storing and querying dense text embeddings
Streamlit	Python library for building interactive web UIs for data/AI applications
AWS EC2	Amazon Web Services Elastic Compute Cloud — cloud virtual machine service used for hosting

Term / Acronym	Definition
AWS S3	Amazon Simple Storage Service — object storage used for documents and datasets
HITL	Human-in-the-Loop — a workflow pattern requiring human review before proceeding
interrupt()	LangGraph API that pauses a workflow and waits for human input before resuming
MemorySaver	LangGraph in-memory checkpointer that persists workflow state across interrupts
EHR/EMR	Electronic Health Record / Electronic Medical Record
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
RFP	Request for Proposal — the capstone project brief issued by IIT Roorkee

1.4 References

- IIT Roorkee Capstone RFP — Request for Proposal, Advanced PG Certification in AI Engineering on Cloud and AIOps, Futureense, 2026
- OpenAI API Documentation — platform.openai.com/docs
- LangGraph Documentation — langchain-ai.github.io/langgraph
- LangChain Documentation — docs.langchain.com
- ChromaDB Documentation — docs.trychroma.com
- Streamlit Documentation — docs.streamlit.io
- AWS EC2 User Guide — docs.aws.amazon.com/ec2

2 Overall Description

2.1 Product Perspective

CareConnect is a standalone AI application built specifically for this capstone. It is not currently integrated with any real-world EHR system, though its architecture is designed to accommodate such integration (via HL7 FHIR APIs) in a future production version.

The system interacts with three external services:

- OpenAI API — for LLM inference (GPT-4o) and text embedding (text-embedding-3-small)
- AWS S3 — for cloud storage of protocol documents and datasets (falls back to local filesystem if unavailable)
- ChromaDB — embedded vector store running within the same process as the application

2.2 Product Functions

At a high level, CareConnect performs the following functions:

#	Function	Description
F1	Patient Data Loading	Reads patient profile, appointment history, and follow-up records from CSV files
F2	RAG Retrieval	Retrieves clinically relevant protocol chunks from ChromaDB based on patient diagnosis
F3	Clinical Summarization	Generates a structured clinical summary and risk classification (low/medium/high/critical)
F4	Risk Routing	Routes the workflow conditionally based on risk level
F5	Reminder Generation	Generates personalized SMS, email, and phone call script reminders
F6	Escalation Report	Generates a clinical escalation report with immediate actions for high-risk patients
F7	Human Approval	Pauses workflow for clinician review and decision (approve/reject/escalate)
F8	Output Formatting	Aggregates and structures the final follow-up plan for display
F9	Dashboard Display	Presents the full follow-up plan in a Streamlit web interface
F10	Session Memory	Maintains conversation history and processed patient log within a session
F11	Batch Processing	Supports processing all patients in the dataset via CLI

2.3 User Classes and Characteristics

User Class	Description	Technical Skill	Primary Use
Care Coordinator	Hospital staff responsible for patient follow-up and scheduling	Low to Medium	Select patients, trigger workflow, review reminders
Clinician / Physician	Doctor who reviews escalated high-risk patients	High to Medium	Review escalation report, approve or reject escalation
System Administrator	IT staff who deploy and maintain the application on AWS	High	Deploy, configure environment, manage S3, manage IAM
Developer (Team 11)	Capstone project developers	High	Extend codebase, tune agents, modify workflow

2.4 Operating Environment

Component	Specification
Cloud Platform	AWS EC2 (t2.medium, Ubuntu 22.04 LTS)
Python Version	3.10 or 3.11 (3.11 recommended)
Web Browser	Google Chrome 110+, Firefox 110+, Edge 110+ (for Streamlit UI)
Network	HTTPS outbound to OpenAI API (api.openai.com) required
Storage	Minimum 2 GB free disk space (for ChromaDB + dependencies)
RAM	Minimum 2 GB (4 GB recommended for smooth Streamlit + ChromaDB)
Local Development	macOS, Windows 10/11, or Linux with Python 3.10+

2.5 Assumptions and Constraints

Assumptions:

- A valid OpenAI API key with GPT-4o access is available
- Patient data used is entirely synthetic — no real patient PII is stored or processed
- The system is for demonstration purposes and is not used in an actual clinical environment
- Internet connectivity is available for OpenAI API calls
- AWS credentials are optional; the system degrades gracefully to local file storage
- The Streamlit frontend is accessed via a modern web browser

Constraints:

- The system processes one patient at a time through the Streamlit dashboard (batch mode available via CLI)
- LLM responses depend on OpenAI service availability — rate limits may cause retried delays
- The current dataset is limited to 20 synthetic patients — production scale would require a real database
- ChromaDB MemorySaver is in-memory — workflow state does not persist across server restarts in the current implementation
- No authentication or role-based access control is implemented in the current version
- No real email or SMS dispatch is implemented — reminders are generated as text output only

3 Functional Requirements

Each requirement is identified by a unique ID in format FR-XX. Priority levels: Critical / High / Medium / Low.

3.1 Patient Data Management

FR-01	Priority: Critical	Requirement: The system shall load patient demographic and clinical data from patients.csv, appointments.csv, and followup_records.csv when a patient ID is provided.	
Rationale: Structured CSV datasets must be present in the ../Dataset/ directory relative to Source_Code/.		Acceptance Criteria: Given a valid patient_id (e.g. P001), the system returns a complete patient_data dict with profile, appointments, and followup_history keys within 1 second.	
FR-02	Priority: High	Requirement: The system shall return a structured error state (not crash) if the patient_id is not found in the dataset.	
Rationale: Graceful error handling is required for robustness.		Acceptance Criteria: Given patient_id='P999', the workflow outputs final_output.status='error' and final_output.message contains 'not found'.	
FR-03	Priority: Medium	Requirement: The system shall support batch processing of all patients in the dataset via CLI with --all flag.	
Rationale: Evaluators may wish to run the system across all patients to demonstrate scalability.		Acceptance Criteria: Running `python main.py --all` processes all 20 patients and prints a summary report.	

3.2 AI Agent Workflow

FR-04	Priority: Critical	Requirement: The system shall execute a LangGraph StateGraph workflow with nodes: load_patient, rag_retrieval, patient_summary, risk_router, reminder, escalation, human_approval, and output.	
Rationale: LangGraph StateGraph is the required orchestration framework per project design.		Acceptance Criteria: Workflow graph compiles without error; all 8 nodes are registered; graph.invoke() completes for any valid patient ID.	
FR-05	Priority: Critical		

Requirement: The Patient Summary Agent shall call the OpenAI GPT-4o model and return a clinical summary (string) and risk_level (one of: low, medium, high, critical) in structured JSON.	
Rationale: Risk stratification is the foundation of the routing logic.	Acceptance Criteria: For patient P001 (AMI), risk_level is 'high' or 'critical'. For patient P004 (post-appendectomy), risk_level is 'low'. Summary is a non-empty string.

FR-06	Priority: Critical
Requirement: The workflow shall route low/medium risk patients to the Reminder Agent and high/critical risk patients to the Escalation Agent, using a LangGraph conditional edge.	
Rationale: Correct routing is essential for clinical safety.	Acceptance Criteria: For 6 critical patients in the dataset, workflow reaches the escalation node. For 2 low patients, workflow skips escalation entirely.

FR-07	Priority: High
Requirement: Each LangGraph node shall check the error state before executing and skip processing gracefully if a prior node has set an error.	
Rationale: Prevents cascading failures from propagating through the workflow.	Acceptance Criteria: When load_patient_node sets error='Patient not found', all downstream nodes return empty dicts without calling the OpenAI API.

3.3 RAG Pipeline

FR-08	Priority: Critical
Requirement: The system shall index documents from ../Synthetic_Documents/ (or AWS S3) into a ChromaDB vector store using OpenAI text-embedding-3-small embeddings on first run.	
Rationale: RAG grounding in clinical protocols is a core differentiator over a pure LLM approach.	Acceptance Criteria: After running `python main.py --rebuild-rag`, ./chroma_db/ directory is created and contains indexed embeddings for all document chunks.

FR-09	Priority: Critical
Requirement: The RAG retrieval node shall query ChromaDB with a diagnosis-specific query and return the top-4 most relevant document chunks as rag_context.	
Rationale: k=4 chunks provide sufficient protocol context without exceeding LLM context window.	Acceptance Criteria: For patient P001 (AMI), rag_context contains text related to cardiac follow-up protocols. Context length is non-zero.

FR-10	Priority: High
--------------	-----------------------

Requirement: The RAG pipeline shall fall back gracefully (return empty string) if ChromaDB is unavailable or returns no results, allowing agents to proceed with a default protocol instruction.	
Rationale: Fallback ensures the system remains functional even if the vector store has an issue.	Acceptance Criteria: With an empty ChromaDB, rag_context is "" and agents produce output using their built-in system prompt only.

FR-11	Priority: Medium
Requirement: The system shall support force-rebuilding the RAG vector store via --rebuild-rag CLI flag or force_rebuild=True parameter.	
Rationale: Document updates should be indexable without manual ChromaDB deletion.	Acceptance Criteria: Running `python main.py --rebuild-rag` deletes the old chroma_db directory and rebuilds from current documents.

3.4 Reminder Generation

FR-12	Priority: Critical
Requirement: The Reminder Agent shall generate exactly three reminders for each patient: one SMS (≤ 160 characters), one Email (with subject and body), and one Phone Call Script.	
Rationale: Three-channel reminders maximize patient reachability per healthcare communication best practices.	Acceptance Criteria: Reminder Agent output contains a reminders list of length 3 with channel values 'SMS', 'Email', and 'Phone Script'.

FR-13	Priority: High
Requirement: All reminders shall be personalized to the patient's name, specific diagnosis, primary physician name, and next follow-up due date.	
Rationale: Personalized reminders have demonstrably higher patient response rates than generic ones.	Acceptance Criteria: SMS reminder contains the patient's name and doctor's name. Email subject references the patient's condition type.

FR-14	Priority: High
Requirement: The SMS reminder shall not exceed 160 characters.	
Rationale: Standard SMS character limit for single-part messages.	Acceptance Criteria: <code>len(sms_reminder_text) <= 160</code> for all 20 test patients.

3.5 Escalation and Human Approval

FR-15	Priority: Critical
--------------	---------------------------

Requirement: The Escalation Agent shall generate a structured escalation report for high/critical risk patients containing: <code>escalation_id</code> , <code>clinical_brief</code> , <code>immediate_actions</code> (list), <code>potential_complications</code> (list), <code>recommended_timeline</code> , and <code>escalation_reason</code> .	
Rationale: Structured escalation reports enable rapid physician triage.	Acceptance Criteria: Escalation report for P003 contains all required fields. <code>immediate_actions</code> is a list with at least 2 items.

FR-16	Priority: Critical
Requirement: The workflow shall pause execution using <code>LangGraph interrupt()</code> when reaching the <code>human_approval</code> node, and shall not proceed until a decision is provided.	
Rationale: Human-in-the-loop is a mandatory requirement in the RFP.	Acceptance Criteria: After triggering escalation for P003, <code>graph.get_state()</code> shows 'human_approval' in <code>snapshot.next</code> . No reminders are generated until <code>resume</code> is called.

FR-17	Priority: Critical
Requirement: The <code>human_approval</code> node shall accept three decisions: 'approve' (route to Reminder Agent), 'reject' (route to Output), 'escalate' (route to Output with urgent flag).	
Rationale: Three decision options cover all clinically relevant responses to an escalation.	Acceptance Criteria: <code>Command(resume='approve')</code> routes to reminder node. <code>Command(resume='reject')</code> routes to output node. Both result in <code>final_output.status='success'</code> .

FR-18	Priority: High
Requirement: The Streamlit frontend shall display the escalation report, clinical summary, and three decision buttons to the clinician when an interrupt is triggered.	
Rationale: The clinician must have full context before making an approval decision.	Acceptance Criteria: When workflow is in 'awaiting_approval' state, the UI shows a red alert panel with escalation details and three active buttons.

3.6 Dashboard and Frontend

FR-19	Priority: Critical
Requirement: The Streamlit frontend shall allow selection of any patient from the 20-patient dataset via a sidebar dropdown.	
Rationale: Patient selection is the primary entry point for the care coordinator.	Acceptance Criteria: All 20 patients appear in the dropdown. Selecting any patient displays their card correctly.

FR - 20	Priority: High
Requirement: The patient card shall display a colour-coded risk badge: green (low), yellow (medium), orange (high), red (critical).	
Rationale: Visual risk coding enables rapid triage in a clinical dashboard.	Acceptance Criteria: P003 (critical) shows a red badge. P004 (low) shows a green badge. Colours match the RISK_COLORS mapping.
FR - 21	Priority: High
Requirement: The frontend shall maintain a session history table showing all patients processed in the current browser session.	
Rationale: Audit trail for the care coordinator's session.	Acceptance Criteria: After processing P001 and P003, the session history table shows 2 rows with correct patient IDs, names, risk levels.
FR - 22	Priority: Medium
Requirement: The frontend shall display a spinner with 'Running AI workflow...' text while the LangGraph workflow is executing.	
Rationale: Prevents the user from thinking the app has frozen during LLM API calls.	Acceptance Criteria: Spinner is visible for the duration of the workflow execution and disappears when results are displayed.

4 Non-Functional Requirements

4.1 Performance

ID	Requirement	Target
NFR-01	Workflow execution time for a single patient (excluding human approvals (type): 8–15 seconds)	< 30 seconds (type)
NFR-02	RAG retrieval latency (ChromaDB similarity search)	< 500 ms
NFR-03	Streamlit page initial load time	< 3 seconds
NFR-04	Batch processing throughput	>= 4 patients per minute
NFR-05	LLM API retry total wait time (3 attempts, exponential backoff)	< 30 seconds before final failure

4.2 Reliability and Availability

- NFR-06: The system shall handle OpenAI rate limit errors without crashing, using tenacity retry with exponential backoff (max 3 attempts, 2s base delay)
- NFR-07: Any single node failure shall not crash the entire workflow — the error shall be captured in state.error and propagated to the output formatter
- NFR-08: The RAG pipeline shall fall back to local document loading if AWS S3 is unreachable
- NFR-09: The application shall be deployable on AWS EC2 and remain accessible without manual intervention after a server reboot (using nohup or a process manager)

4.3 Security

- NFR-10: The OpenAI API key shall be stored in a .env file and never hardcoded in source files or committed to version control
- NFR-11: AWS credentials shall use IAM role-based access (EC2 instance profile) rather than hardcoded keys
- NFR-12: All patient data in this project is synthetic — no real PII shall be used or stored
- NFR-13: The Streamlit app port (8501) shall be restricted by AWS Security Group to authorised IP ranges in production
- NFR-14: The .env file shall be listed in .gitignore and excluded from all submissions

4.4 Usability

- NFR-15: A non-technical care coordinator shall be able to generate a follow-up plan within 3 clicks without reading documentation
- NFR-16: Risk levels shall be communicated with both colour coding and text labels to ensure accessibility
- NFR-17: All reminder output shall use plain, patient-friendly language — no unexplained medical jargon

- NFR-18: The HOW_TO_RUN.md guide shall enable a developer to set up and run the project locally within 15 minutes

4.5 Maintainability

- NFR-19: Each agent (patient_summary_agent.py, reminder_agent.py, escalation_agent.py) shall be independently testable without running the full LangGraph workflow
- NFR-20: The LLM model name (MODEL_NAME) and embedding model (EMBEDDING_MODEL) shall be configurable via config.py without code changes
- NFR-21: Adding a new document to the RAG knowledge base shall require only copying a .txt file to Synthetic_Documents/ and running --rebuild-rag
- NFR-22: All functions using OpenAI API calls shall have the @retry decorator applied for consistent error handling

5 System and Data Requirements

5.1 Hardware Requirements

Component	Minimum	Recommended
CPU	2 vCPU	4 vCPU
RAM	2 GB	4 GB
Disk	5 GB free	10 GB free
Network	1 Mbps	10 Mbps (for LLM API latency)
AWS EC2	t2.micro	t2.medium

5.2 Software Dependencies

Package	Version	Purpose
openai	>=1.50.0	GPT-4o inference and text-embedding-3-small
langgraph	>=0.2.28	Multi-agent workflow orchestration, interrupt, MemorySaver
langchain	>=0.3.0	Document loaders, text splitters, retrieval chain
langchain-openai	>=0.2.0	LangChain OpenAI embeddings integration
langchain-chroma	>=0.1.4	LangChain ChromaDB vector store integration
chromadb	>=0.5.0	Persistent vector database for RAG
streamlit	>=1.38.0	Web dashboard frontend
pandas	>=2.0.0	CSV dataset loading and manipulation
boto3	>=1.35.0	AWS S3 client for document retrieval
python-dotenv	>=1.0.0	Environment variable management from .env
pydantic	>=2.0.0	Data validation for agent outputs
tenacity	>=8.5.0	Retry logic with exponential backoff

5.3 Data Requirements

5.3.1 Input Dataset Files

File	Format	Records	Required Fields
patients.csv	CSV	20	patient_id, name, age, gender, diagnosis, medications, risk_level, next_followup
appointments.csv	CSV	40	appointment_id, patient_id, appointment_date, appointment_type, status
followup_records.csv	CSV	40	followup_id, patient_id, followup_date, contact_method, outcome, reminder_sent

5.3.2 RAG Document Files

- `medical_followup_protocols.txt` — Clinical follow-up protocols covering 7 medical domains (300+ lines). Loaded at startup for RAG indexing.
- `patient_discharge_summaries.txt` — Sample discharge summaries for 6 critical patients. Provides discharge-level context for agent reasoning.
- Format: Plain UTF-8 text (.txt). Location: `../Synthetic_Documents/` relative to `Source_Code/`

5.3.3 Generated Data

- ChromaDB vector store — persisted to `./Source_Code/chroma_db/` after first indexing run
- LangGraph state snapshots — held in `MemorySaver` (in-memory, per-session)
- Session history — held in Streamlit `session_state` (browser session only, not persisted to disk)

6 Use Cases

UC-01: Generate Follow-Up Plan for Low/Medium Risk Patient

Actor	Care Coordinator
Main Flow	1. Coordinator selects a patient (e.g., P004 — low risk) from the sidebar 2. Coordinator clicks 'Generate Follow-Up Plan' 3. System loads patient data from CSV 4. System retrieves relevant protocol chunks from ChromaDB 5. Patient Summary Agent generates summary (risk_level = 'low') 6. Risk Router sends workflow to Reminder Agent 7. Reminder Agent generates SMS, Email, and Phone Script 8. Output is displayed: risk badge, summary, 3 reminder cards
Preconditions	Patient data exists in CSV. ChromaDB is initialized. OPENAI_API_KEY is valid.
Postconditions	Follow-up plan displayed within 30 seconds. 3 reminders generated.

UC-02: Handle Critical Patient with Human Approval

Actor	Care Coordinator + Clinician
Main Flow	1. Coordinator selects P003 (Congestive Heart Failure — critical) 2. Coordinator clicks 'Generate Follow-Up Plan' 3. System runs summary (risk_level = 'critical') 4. Escalation Agent generates clinical escalation report 5. Workflow pauses at Human Approval node (interrupt()) 6. Clinician reviews escalation details on approval panel 7. Clinician clicks 'Approve' — workflow resumes 8. Reminder Agent generates personalized reminders 9. Final output displayed with escalation report + reminders
Preconditions	Same as UC-01. Patient P003 has risk_level='critical' in dataset.
Postconditions	Interrupt pauses correctly. Reminders generated only after approval. human_decision='approve' in output.

UC-03: Reject Escalation

Actor	Clinician
Main Flow	1. Escalation triggered for high-risk patient (steps 1–6 as UC-02) 2. Clinician reviews case and determines no immediate action needed 3. Clinician clicks 'Reject' 4. Workflow routes to Output Formatter without generating reminders 5. Output shows escalation_required=True, human_decision='reject', reminders=[]
Preconditions	Same as UC-02.

Postconditions	No reminders generated. final_output.human_decision='reject'.
-----------------------	---

UC-04: Rebuild RAG Vector Store

Actor	System Administrator
Main Flow	1. Admin adds a new protocol document to Synthetic_Documents/ 2. Admin runs: python main.py --rebuild-rag 3. System deletes old chroma_db/ and rebuilds from all documents 4. New document chunks are searchable in subsequent patient queries
Preconditions	New .txt file exists in Synthetic_Documents/. OPENAI_API_KEY is valid.
Postconditions	chroma_db/ recreated. Chunks from new document appear in RAG retrieval results.

UC-05: Batch Process All Patients

Actor	Developer / Administrator
Main Flow	1. User runs: python main.py --all --limit 5 2. System processes patients P001 through P005 sequentially 3. For high/critical patients, human input is requested via CLI prompt 4. Summary printed after completion: X/5 successful, Y escalations
Preconditions	All CSV files present. OPENAI_API_KEY valid.
Postconditions	All 5 patients processed. CLI shows success count and escalation count.

7 User Stories

User stories follow the format: As a [role], I want to [action] so that [benefit]. Each is tagged with a requirement ID.

ID	As a...	I want to...	So that...	FR Ref
US-01	Care Coordinator	select any patient from a searchable dropdown	and quickly find the patient I need to follow up on	FR-19
US-02	Care Coordinator	see a colour-coded risk badge on the patient card	and immediately know which patients need urgent attention	FR-20
US-03	Care Coordinator	click a single button to generate a complete At-Risk list for manually look up protocols		FR-05
US-04	Care Coordinator	receive three ready-to-send reminders (SMS, email, the patient's phone)	and follow up with a single click	FR-12, FR-13
US-05	Clinician	receive a structured escalation report when a medical patient is declined	and quickly	FR-15, FR-16
US-06	Clinician	approve, reject, or escalate a flagged patient case with one click	and the case workflow proceeds with the next step	FR-17, FR-18
US-07	Care Coordinator	see a session history of all patients I have processed	and my workday without paper notes	FR-21
US-08	System Administrator	rebuild the AI knowledge base by placing clinical protocols into a federated file	and the system immediately	FR-22
US-09	Developer	have each agent defined in its own file with a clear prompt and contracting	and the workflow orchestration	FR-19
US-10	Care Coordinator	have the system recover gracefully if the AI service is temporarily unavailable		FR-06, FR-07

8 Integration Requirements

8.1 OpenAI API Integration

- Endpoint: api.openai.com (HTTPS)
- Authentication: Bearer token (OPENAI_API_KEY in .env)
- Models used: gpt-4o (chat completions), text-embedding-3-small (embeddings)
- All chat completion calls use response_format={'type': 'json_object'} for structured output
- Rate limit handling: tenacity @retry decorator with stop_after_attempt(3) and wait_fixed(2)
- Timeout: OpenAI SDK default (60s); configurable via httpx_client if needed

8.2 AWS S3 Integration

- Client: boto3 s3.client with IAM role credentials
- Bucket: careconnect-documents
- Document prefix: documents/ | Dataset prefix: datasets/
- Triggered at startup in initialize_rag() — downloads .txt files to temp files for indexing
- Falls back to local ../Synthetic_Documents/ if S3 fails
- No S3 writes are performed by the application in the current version

8.3 ChromaDB Integration

- Mode: Embedded (in-process) — no separate server required
- Persistence: ./chroma_db/ directory (relative to Source_Code/)
- Embedding function: OpenAIEmbeddings(model='text-embedding-3-small')
- Collection: default (auto-named by LangChain)
- Operations: from_documents() (build), similarity_search(query, k=4) (retrieve)

8.4 LangGraph Interrupt / Resume Integration

- graph.stream(initial_state, config=config) — starts the workflow
- graph.get_state(config) — checks whether workflow is paused at an interrupt
- snapshot.next — list of next nodes; contains 'human_approval' when interrupted
- snapshot.tasks[0].interrupts[0].value — retrieves the interrupt payload (escalation data)
- graph.stream(Command(resume=decision), config=config) — resumes from the interrupt
- Thread isolation: each patient run uses a unique thread_id (UUID4) stored in Streamlit session_state

9 Constraints and Limitations

Constraint	Category	Description
Synthetic Data Only	Data	All patient data is entirely synthetic. No real patient PHI/PII is used. The system must not store or process real patient data.
No Real Message Dispatch	Integration	Reminders are generated as text but not sent via SMS (Twilio) or email (SES). Real message dispatch is not implemented.
Single-Patient Dashboard	Performance	The Streamlit dashboard processes one patient at a time. Parallel multi-patient processing is not supported.
In-Memory State Only	Persistence	LangGraph MemorySaver stores state in RAM. State is lost on server restart. A PostgreSQL database is not used for state persistence.
No Auth/RBAC	Security	No authentication or role-based access control. Any user with the EC2 URL can access the application.
OpenAI API Dependency	Availability	All AI functionality requires a valid OpenAI API key and internet access. Offline or self-hosted models are not used.
ChromaDB Local Only	Scalability	ChromaDB runs embedded. For horizontal scaling (multiple EC2 instances), a remote vector database is not used.
4-Week Timeline	Scope	The project was delivered in 4 weeks as per capstone requirements. Production hardening and additional features are out of scope.

10 Glossary

Term	Definition
Agent	An autonomous AI component within LangGraph that takes inputs, calls an LLM or tool, and produces outputs
Checkpoint	LangGraph mechanism for persisting workflow state between nodes and across interrupts
Chunk	A fixed-size segment of text produced by splitting a document, stored as a vector in ChromaDB
Conditional Edge	A LangGraph edge that routes the workflow to different nodes based on a function's return value
Embedding	A dense numerical vector representation of text, used for semantic similarity search in ChromaDB
Follow-up	A scheduled communication with a patient after hospital discharge to monitor recovery and medication adherence
interrupt()	LangGraph function that pauses a workflow, exposes data to the calling application, and waits for a resume signal
Node	A single step in the LangGraph workflow, implemented as a Python function that reads and updates the state
Risk Level	A clinical classification (low/medium/high/critical) assigned to a patient based on their diagnosis, vital signs, and history
StateGraph	LangGraph's primary workflow class — a directed graph of nodes with a shared typed state dictionary
MemorySaver	LangGraph's in-memory checkpointer — stores full state snapshots for each step, enabling interruption and resumption
thread_id	A unique identifier (UUID) for each patient workflow run, used to isolate state between different patients
Vector Store	A database optimised for storing and querying dense vector embeddings (e.g., ChromaDB)
workflow_phase	Streamlit session_state variable tracking the UI state: idle / processing / awaiting_approval / completed